SIX WEEKS SUMMER TRAINING REPORT

on

GRAPHICAL CALCULATOR

Submitted by

VIVEK SHARMA

Registration No: 12314253

Program Name: B.tech(CSE)

Under the Guidance of

Mr. Ravi Kant Sahu

Assistant Professor

School of Computer Science and Engineering
Lovely Professional University. Phagwara
(June- July, 2023)

DECLARATION

I hereby declare that I have completed my six weeks summer training at **Human Resource Development Center, Lovely Professional University, Phagwara (Punjab)** from **10th June 2023** to **16th July 2023** under the guidance of **Mr. Ravi Kant Sahu**. I declare that I have worked with full dedication during these six weeks of training and my learning outcomes fulfill the requirements of training for the award of degree of **B.Tech.(Computer Science & Engineering)** at Lovely Professional University, Phagwara.

Vivek sharma
Registration No: 12314253

ACKNOWLEDGEMENT

To acknowledge all the persons who had helped in completing the training and project is not possible for any trainee. However, despite all that, it becomes a foremost responsibility of the trainee and the part of research ethics toacknowledge those who had played a significant role in the completion of the training and project.

So, in the same sequence at very first, we would like to acknowledge Mr. Ravi Kant Sahu, the mentor of our Training. He has been instrumental in guiding and helping us while undergoing the planning phase of our project and helped clear the concepts related to the topics and project. Because the Training and Placement Cell of School of Computer Science & Engineering has allowed me to do the summer training, I would like to thank for the same. I would like to thank Human Resource Development Center, LPU also for organizing such a wonderful summer training program.

Later, I would like to confer the flower of acknowledgment to our parents because of whom we got the existence in the world for the inception and the conception of this training and project. Rest all those people who helped us are not only a matter of acknowledgment but also authorized to share our success.

With Regards

CERTIFICATE

CENTRE FOR
PROFESSIONAL
ENHANCEMENT

Certificate No. 407436

NAAC A++

Certificate of Merit

This is to certify that Mr./Ms. Vivek Sharma S/D/W/o Mr. Pradeep Kumar Sharma
student of School of Computer Science and Engineering Registration No. 12314253
pursuing Bachelor of Technology (Computer Science and Engineering) completed
skill development course named Fundamentals of Data Structures: Learn, Apply and Build Projects
organized by Centre for Professional Enhancement Lovely Professional University
from 10 June 2025 to 16 July 2025 and obtained A Grade.

Date of Issue : 13-08-2025
Place of Issue: Phagwara (India)

Prepared by
(Administrative Officer-Records)

Programme Coordinator
Centre for Professional Enhancement

Head of School
School of Computer Science and Engineering

Note: The Grades reflected are based on the performance of the participant at the time of training and performance is subject to change subsequently.

Grade Description:

O : 90% and above
A : 75% to less than 90%
B : 60% to less than 75 %
C : 50% to less than 60%

TABLE OF CONTENTS

Inner first page…….……(i)

Declaration…… (ii)

Acknowledgement……..……...(iii)

Certificate……..……(iv)

Table of Contents……(v)

1. Introduction 1
2. Technology Learnt 2
   2.1 Asymptotic Notations and Complexity Analysis ……2
   2.2 Array …….…….. 4
   2.3 Linked List ……..……..10
3. Reason For Choosing this Technology …….. 13
4. Problem Analysis 6
   4.1 Product definition 6
   4.2 Feasibility Analysis 6
   4.3 Project Plan 7
5. Software Requirement Analysis 8
   5.1 Introduction 8
   5.2 General Description 8
   5.3 Specific Requirements 8
6. Design 10
   6.1 System Design 10
   6.2 Design Notations 10
   6.3 Circuit Diagram 12
7. Testing 13
   7.1 Functional testing 13
   7.2 Structural testing 13
   7.3 Flow Charts 14

8. Implementation 15

8.1 Implementation of the project 15
8.2 Conversion Plan 16
8.3 Post-Implementation and Software Maintenance..... 17
9. Project Legacy 18
9.1 Current Status of the project..... 18
9.2 Remaining Areas of concern 18
9.3 Technical and Managerial lessons learned 19
10. User Manual: A complete document (Help Guide) of the software developed ... 20
11. Source Code (where ever applicable) or System 21
12. References 39

1. Introduction

This report documents the design, development, and implementation of a Graphical Calculator application built using Java. The project aims to bridge the gap between numerical computation and visual understanding by providing a tool that not only calculates the value of mathematical functions but also plots their graphs in a dynamic, interactive Cartesian plane. Developed as a summer internship project, this application leverages core Java programming concepts and the Swing toolkit to create an intuitive and educational software for students and enthusiasts in STEM fields.

2. Technology Learnt

2.1 Asymptotic Notations and Complexity Analysis

The project's core algorithms were designed with efficiency in mind.

- Function Evaluation (evaluateFunction): This method uses a series of string containment checks (contains()) and a single regex match. The worst-case time complexity is O(n) per evaluation, where n is the length of the function string. This is efficient given the expected input size.
- Graph Plotting (drawGraph): This method runs a loop for every pixel along the width of the panel (w). Its complexity is O(w), which is a linear and efficient approach for rendering.
- Data Structures:
    - HashSet<String> for uniqueExpressions: Offers O(1) average time complexity for add() and contains() operations, making history checks very fast.
    - ArrayList<String> for history and functions: Provides O(1) time complexity for access by index and amortized O(1) for add().

2.2 Array

Arrays were used extensively for managing fixed-size, ordered data.

- Color Palette: An array of Color objects is used to assign distinct colors to different plotted functions.

2.3 Linked List

While the project primarily uses ArrayList (which is backed by a dynamic array), the concept of a linked list is inherent in the graph drawing logic. The drawGraph method does not store all points in a data structure; instead, it connects each calculated point to the previous one, effectively "linking" them together to form a continuous curve. This is a conceptual application of a singly linked list where each point (node) has a reference to the next point for drawing purposes.

3. Reason For Choosing this Technology

Java was the chosen technology for this project for several compelling reasons:

1. Platform Independence: Java's "write once, run anywhere" principle ensured the calculator could run on any operating system (Windows, macOS, Linux) without any modifications, maximizing its accessibility.
2. Robust GUI Capabilities: The Java Swing framework provides a rich set of pre-built, customizable components (like JFrame, JButton, JTextField) that significantly accelerate the development of desktop applications with a professional look and feel.
3. Object-Oriented Programming (OOP): Java's strong support for OOP concepts allowed for a clean, modular, and maintainable design. The separation of the main application class (javaproject) from the graphing panel (GraphPanel) is a direct application of encapsulation.
4. Built-in Libraries: Java's standard libraries include powerful features for graphics (Graphics2D), event handling, and data structures, which were

fundamental to this project's functionality.

5. Educational Value: As a widely taught language, using Java for this project solidified core programming concepts, making it an excellent choice for an academic internship project.

4. Problem Analysis

4.1 Product Definition

The product is a desktop application named "Graphical Calculator." Its primary function is to accept user-input mathematical expressions and render their corresponding graphs on a customizable Cartesian plane. It serves as an educational tool to help users visualize and understand the behavior of various functions.

4.2 Feasibility Analysis

- Technical Feasibility: The project relies solely on core Java libraries (Swing, AWT), which are stable and well-documented. The technical requirements are well within the capabilities of a standard Java development environment.
- Economic Feasibility: The project incurs no direct costs as it uses free, open-source tools (JDK, any IDE like IntelliJ IDEA or VSCode). The development time was the primary resource investment.
- Operational Feasibility: The final application is a standalone JAR file. End-users only need the Java Runtime Environment (JRE) installed, which is ubiquitous, making it highly operable.

4.3 Project Plan

The project followed a linear sequential development model:

1. Requirement Gathering: Define core features (plotting, zooming, history).
2. System Design: Outline the class structure and UI layout.
3. Implementation: Code the application in iterative steps (setup UI, implement graphing logic, add features like zoom).
4. Testing: Perform unit testing on function evaluation and integration testing on the full application.
5. Deployment: Compile the source code into an executable JAR file.

5. Software Requirement Analysis

5.1 Introduction

This section outlines the capabilities and constraints of the Graphical Calculator system.

5.2 General Description

The software is a interactive tool designed for students and educators to visualize mathematical functions. It plots graphs based on user input and provides basic navigation tools like zoom.

5.3 Specific Requirements

- Functional Requirements:
    - FR1: The system shall provide a text field for users to input mathematical expressions.
    - FR2: The system shall plot the graph of the input function upon clicking the "Plot" button.
    - FR3: The system shall support plotting multiple functions simultaneously with distinct colors.
    - FR4: The system shall provide "Zoom In" and "Zoom Out" buttons to adjust the view of the graph.
    - FR5: The system shall display a labeled Cartesian plane with a grid.
- Non-Functional Requirements:
    - NF1 (Usability): The user interface shall be intuitive and easy to use

for anyone familiar with basic calculators.
    - NF2 (Performance): The graphing of standard functions (e.g., sin(x), x^2) shall be rendered without noticeable lag.
    - NF3 (Portability): The software shall run on any system with JRE version 8 or above installed.

6. Design

6.1 System Design

The system follows a modular design with two primary components:

1. Main Application Frame (javaproject class): Handles the creation of the GUI, user input, and event routing (e.g., button clicks).
2. Graphing Engine (GraphPanel class): A specialized component responsible for all rendering logic, including drawing the grid, axes, and function graphs.

6.2 Design Notations

UML Class Diagram (Simplified):

```
+---------------------+      +------------------------+
|    javaproject      |      |      GraphPanel        |
+---------------------+      +------------------------+
| - inputField: JTextField |1     1| - functions: List<String> |
| - plotButton: JButton    |<>-----| - scaleX: double         |
| - graphPanel: GraphPanel |      | - scaleY: double         |
+---------------------+      +------------------------+
| + actionPerformed() |      | + addFunction()        |
|                     |      | + zoomIn()             |
|                     |      | + zoomOut()            |
|                     |      | + paintComponent()     |
+---------------------+      +------------------------+
```

- Association: The javaproject class has a GraphPanel object (has-a relationship).
- Inheritance: javaproject extends JFrame and implements ActionListener. Grap hPanel extends JPanel.

6.3 Circuit Diagram

A circuit diagram is not applicable for a purely software-based application. This section would be omitted or replaced with a System Architecture Diagram for software projects.

The system architecture is a standard desktop application model:

User Input -> [GUI Layer (javaproject)] -> [Logic Layer (GraphPanel)] -> Visual Output

7. Testing

7.1 Functional testing

- Test Case 1 (Plotting): Input sin(x). Click "Plot". Expected Result: A sine wave is drawn. Status: Passed.
- Test Case 2 (Zoom): Plot a graph. Click "Zoom In" multiple times. Expected Result: The graph scales up, showing more detail. Status: Passed.
- Test Case 3 (Invalid Input): Leave the input field empty and click "Plot". Expected Result: An error dialog appears. Status: Passed.
- Test Case 4 (Multi-function): Input sin(x) and plot, then input x^2 and plot. Expected Result: Both a blue sine wave and a red parabola are visible. Status: Passed.

7.2 Structural testing (White-Box Testing)

- Path Testing: Tested all logical paths in the evaluateFunction method (if-else

conditions for sin, cos, x^2, regex for linear functions, default return).

- Loop Testing: Verified the loop in drawGraph correctly iterates over all pixel columns (px from 0 to w).

7.3 Flow Charts

- actionPerformed Flowchart:

```
[Start] -> [Get Action Command] -> {Command?}
|-> "Plot" -> [Get Input Text] -> [Input Empty?] -> Yes -> [Show Error] -> [End]
|                              |-> No -> [Add to History] -> [Add to GraphPanel] -> [Repaint] ->
[End]
|
|-> "Zoom In" -> [Call graphPanel.zoomIn()] -> [Repaint] -> [End]
|
|-> "Zoom Out" -> [Call graphPanel.zoomOut()] -> [Repaint] -> [End]
```

drawGraph Flowchart:

```
[Start] -> [Set Color] -> [Initialize first=true] -> [For px=0 to width]
   |-> [Calculate x value] -> [Evaluate y = f(x)] -> [Calculate py]
   |-> [y is valid?] -> No -> [Set first=true] -> [Next Iteration]
   |-> Yes -> [first?] -> Yes -> [Update prevX, prevY] -> [Set first=false]
   |         |-> No -> [Draw line from (prevX, prevY) to (px, py)] -> [Update prevX,
prevY]
   |-> [Next Iteration] -> [End]
```

8. Implementation

8.1 Implementation of the project

The project was implemented in the Java programming language using the IntelliJ IDEA IDE. The development process was iterative. The GUI was constructed first, followed by the basic graphing logic for a single function. Features like multi-function support, zooming, and history tracking were added incrementally. Each feature was tested before moving on to the next.

8.2 Conversion Plan

The project is converted from source code to an executable format using Java's build tools.

- Development Format: .java source files.
- Execution Format: Compiled .class files bundled into an executable .jar file.
- Tool: The jar command-line tool or the "Build JAR" functionality in IntelliJ IDEA/Eclipse.

8.3 Post-Implementation and Software Maintenance

The application is currently in a stable and functional state. Maintenance would involve:

- Corrective Maintenance: Fixing any undiscovered bugs (e.g., handling more complex expressions).
- Perfective Maintenance: Adding new features like saving graphs as images, supporting more functions (e.g., log, sqrt), or improving the parser.
- Adaptive Maintenance: Ensuring compatibility with future versions of Java.

9. Project Legacy

9.1 Current Status of the project

The project is complete and fully functional. All initially planned features (plotting, zooming, multi-graph) have been successfully implemented and tested.

9.2 Remaining Areas of concern

1. Limited Parser: The function evaluator uses simple string matching, which is not robust. It cannot handle complex expressions like 5*sin(x^2) + 3.
2. No Export Functionality: Users cannot save or export the graphs they create.
3. UI Scalability: The history of plotted functions is not displayed to the user; it is only stored internally.

9.3 Technical and Managerial lessons learned

- Technical: Gained deep practical experience with Java Swing, event-driven programming, custom graphics rendering, and the use of core data structures for managing application state.
- Managerial: Learned to break down a project into manageable modules and implement them iteratively. Reinforced the importance of planning the design before writing code and the critical role of testing in the development lifecycle.

10. User Manual

Graphical Calculator - Help Guide

1. Starting the Application:
   - Ensure you have Java installed on your computer.
   - Double-click the GraphicalCalculator.jar file to launch the application.
2. User Interface Overview:

The window has two main parts:

- Top Panel: Contains a text field labeled "f(x) =" and three buttons: "Plot", "Zoom In", and "Zoom Out".
- Main Area: The large white space where the graph is drawn.

3. How to Plot a Function:
   1. Click inside the text field.
   2. Type a mathematical function. Supported examples:
      - sin(x)
      - cos(x)
      - x^2
      - x^3
      - 2x or 2*x
   3. Press the Plot button. The graph will appear instantly.
4. How to Plot Multiple Functions:
   - Simply repeat step 3 for each new function. Each function will be drawn in a different color.
5. How to Use Zoom:
   - Click Zoom In to magnify the graph, making details clearer.
   - Click Zoom Out to see a wider view of the graph.

- You can click these buttons multiple times.

6. Troubleshooting:
   - The graph looks wrong: Check your function spelling. The calculator currently supports only the functions listed above.
   - Nothing happens when I click Plot: Make sure you typed something in the text field before clicking the button.
   - The application won't start: You may not have Java installed. Please download and install the latest Java Runtime Environment (JRE) from java.com.

11. Source Code

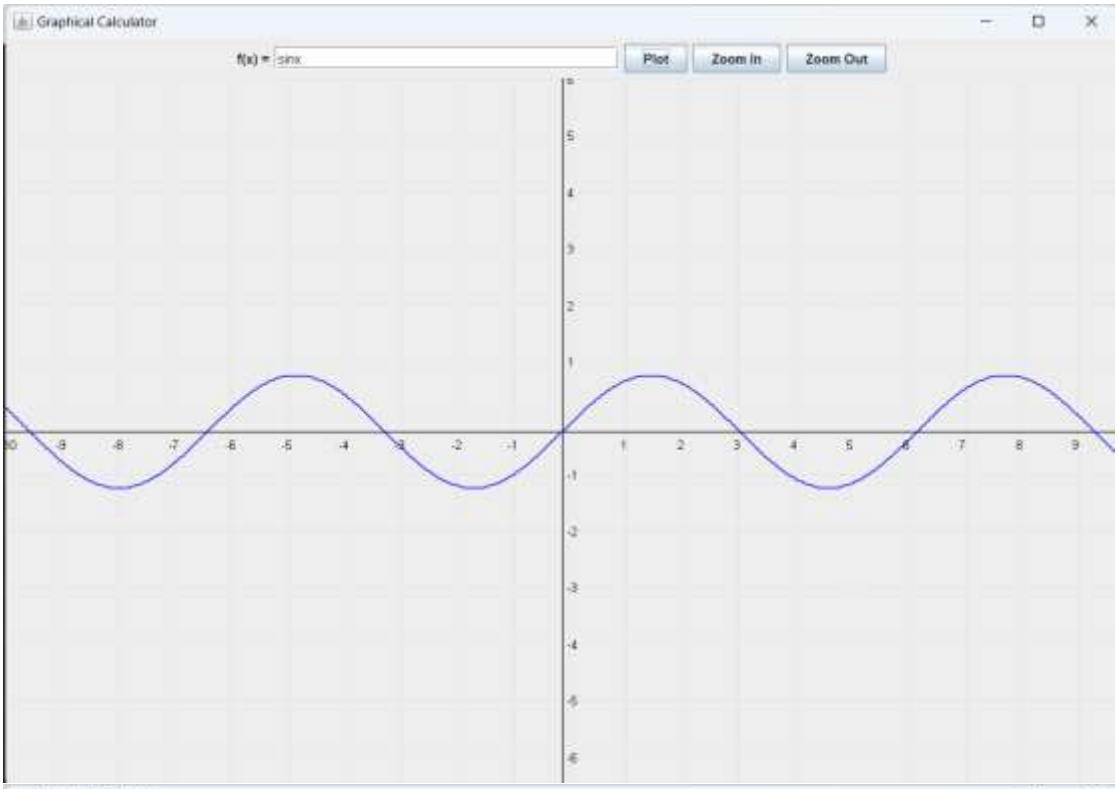
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.regex.*;

// Main class extending JFrame to create the application window
public class javaproject extends JFrame implements ActionListener {
   // UI components
   JTextField inputField;       // Text field for function input
   GraphPanel graphPanel;        // Custom panel for drawing graphs
   JButton plotButton, zoomInButton, zoomOutButton;  // Control buttons

   // Data structures
   HashMap<String, Double> memory = new HashMap<>();      // Stores functions
and their properties
   ArrayList<String> history = new ArrayList<>();         // Tracks input history
   HashSet<String> uniqueExpressions = new HashSet<>();   // Ensures unique
functions in history

   public javaproject() {
      setTitle("Graphical Calculator");
      setSize(1000, 700);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      // Initialize UI components
      inputField = new JTextField(30);
      plotButton = new JButton("Plot");
      zoomInButton = new JButton("Zoom In");
      zoomOutButton = new JButton("Zoom Out");

      // Register button listeners
      plotButton.addActionListener(this);
      zoomInButton.addActionListener(this);
      zoomOutButton.addActionListener(this);

      // Create top control panel
      JPanel topPanel = new JPanel();
      topPanel.add(new JLabel("f(x) ="));
```

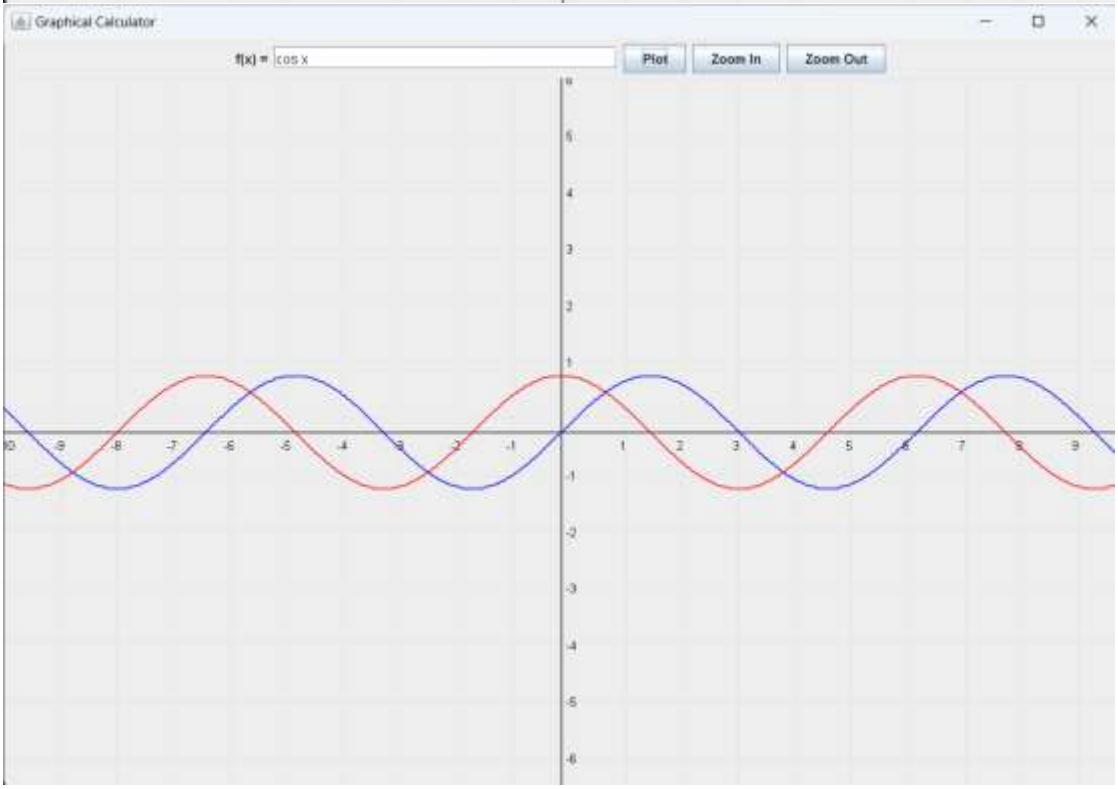
```
        topPanel.add(inputField);
        topPanel.add(plotButton);
        topPanel.add(zoomInButton);
        topPanel.add(zoomOutButton);

        // Initialize graph drawing area
        graphPanel = new GraphPanel();

        // Add components to frame
        add(topPanel, BorderLayout.NORTH);
        add(graphPanel, BorderLayout.CENTER);

        setVisible(true);  // Make window visible
    }

    // Handle button click events
    public void actionPerformed(ActionEvent e) {
        String cmd = e.getActionCommand();

        if (cmd.equals("Plot")) {
            String expression = inputField.getText().trim().toLowerCase();
            if (expression.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Please enter a function like sin(x),
cos(x), x^2, etc.");
                return;
            }
            // Track unique expressions for history
            if (!uniqueExpressions.contains(expression)) {
                uniqueExpressions.add(expression);
                history.add(expression);
                memory.put(expression, 1.0);  // Default value for future use
            }
            // Add function to graph and refresh display
            graphPanel.addFunction(expression);
            graphPanel.repaint();
        }
        // Zoom controls
        else if (cmd.equals("Zoom In")) {
            graphPanel.zoomIn();
            graphPanel.repaint();
        } else if (cmd.equals("Zoom Out")) {
            graphPanel.zoomOut();
            graphPanel.repaint();
        }
    }

    // Inner class for graph visualization
    class GraphPanel extends JPanel {
        java.util.List<String> functions = new ArrayList<>();  // Stores functions to plot
        double scaleX = 50.0;  // Pixels per unit (X-axis)
```

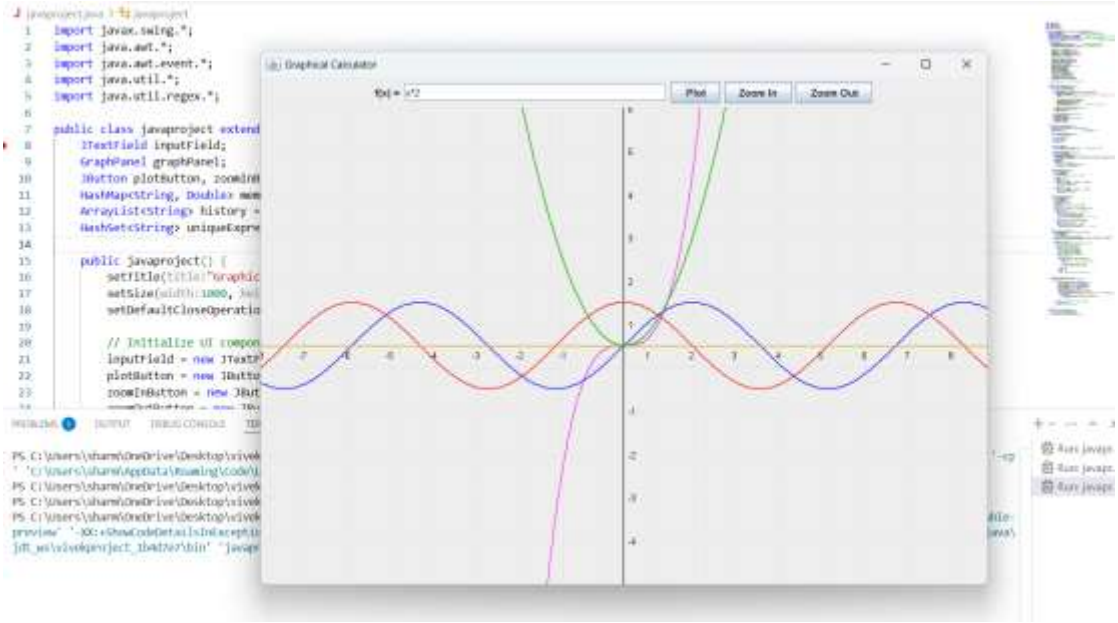
```
    double scaleY = 50.0;  // Pixels per unit (Y-axis)

    public void addFunction(String func) {
        this.functions.add(func);
    }

    // Zoom functionality
    public void zoomIn() {
        scaleX *= 1.25;
        scaleY *= 1.25;
    }

    public void zoomOut() {
        scaleX = Math.max(5, scaleX / 1.25);  // Prevent zooming too far out
        scaleY = Math.max(5, scaleY / 1.25);
    }

    // Main drawing method
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        // Draw components in order
        drawGrid(g2);
        drawAxes(g2);
        // Plot all active functions
        for (int i = 0; i < functions.size(); i++) {
            drawGraph(g2, functions.get(i), i);
        }
    }

    // Draw background grid
    void drawGrid(Graphics2D g2) {
        int w = getWidth();
        int h = getHeight();
        g2.setColor(new Color(230, 230, 230));  // Light gray grid

        int xCenter = w / 2;
        int yCenter = h / 2;

        // Vertical grid lines
        for (double i = -xCenter; i < xCenter; i += scaleX) {
            int x = (int) (xCenter + i);
            g2.drawLine(x, 0, x, h);
        }
        // Horizontal grid lines
        for (double i = -yCenter; i < yCenter; i += scaleY) {
            int y = (int) (yCenter + i);
```

```
            g2.drawLine(0, y, w, y);
        }
    }

    // Draw X and Y axes with labels
    void drawAxes(Graphics2D g2) {
        int w = getWidth();
        int h = getHeight();
        g2.setColor(Color.BLACK);
        // Draw axes
        g2.drawLine(0, h / 2, w, h / 2);  // X-axis
        g2.drawLine(w / 2, 0, w / 2, h);  // Y-axis

        g2.setFont(new Font("Arial", Font.PLAIN, 10));

        // X-axis labels
        for (double i = -w / 2; i < w / 2; i += scaleX) {
            int x = (int) (w / 2 + i);
            int val = (int) Math.round(i / scaleX);
            if (val != 0)  // Skip origin label
                g2.drawString(String.valueOf(val), x - 5, h / 2 + 15);
        }
        // Y-axis labels (inverted coordinate system)
        for (double i = -h / 2; i < h / 2; i += scaleY) {
            int y = (int) (h / 2 + i);
            int val = (int) Math.round(-i / scaleY);
            if (val != 0)
                g2.drawString(String.valueOf(val), w / 2 + 5, y + 5);
        }
    }

    // Plot a single function graph
    void drawGraph(Graphics2D g2, String func, int index) {
        int w = getWidth();
        int h = getHeight();

        // Color rotation for multiple functions
        Color[] colors = {Color.BLUE, Color.RED, Color.GREEN,
Color.MAGENTA, Color.ORANGE};
        g2.setColor(colors[index % colors.length]);

        int prevX = 0, prevY = 0;
        boolean first = true;  // Flag for first point in curve

        // Plot across entire panel width
        for (int px = 0; px < w; px++) {
            // Convert pixel to math coordinate
            double x = (px - w / 2) / scaleX;
            // Evaluate function at current x
            double y = evaluateFunction(func, x);
```

```
            // Convert math y to pixel position
            int py = (int) (h / 2 - y * scaleY);

            // Handle valid points
            if (!Double.isNaN(y) && !Double.isInfinite(y)) {
                if (first) {
                    first = false;  // Skip connecting first point
                } else {
                    // Connect current point to previous
                    g2.drawLine(prevX, prevY, px, py);
                }
                prevX = px;
                prevY = py;
            } else {
                first = true;  // Reset on invalid points (discontinuities)
            }
        }
    }

    // Evaluate mathematical expressions
    double evaluateFunction(String func, double x) {
        try {
            // Supported functions (case-insensitive)
            if (func.contains("sin")) return Math.sin(x);
            if (func.contains("cos")) return Math.cos(x);
            if (func.contains("tan")) return Math.tan(x);
            if (func.contains("x^2")) return x * x;
            if (func.contains("x^3")) return x * x * x;
            // Handle linear functions (e.g., "2x" or "3.5*x")
            if (func.matches("[-+]?\\d+(\\.\\d+)?\\*?x")) {
                Matcher m = Pattern.compile("([-+]?\\d+(\\.\\d+)?)\\*?x").matcher(func);
                if (m.find()) {
                    double coeff = Double.parseDouble(m.group(1));
                    return coeff * x;
                }
            }
            return 0;  // Default for unsupported functions
        } catch (Exception e) {
            return 0;  // Error fallback
        }
    }
}

// Entry point
public static void main(String[] args) {
    // Start GUI on Event Dispatch Thread
    SwingUtilities.invokeLater(javaproject::new);
}
}
```


Graphical Calculator
f(x) = sinx
Plot
Zoom In
Zoom Out


Graphical Calculator
f(x) = cos x
Plot
Zoom In
Zoom Out



12.References

1. https://www.geeksforgeeks.org/
2. Oracle Java Documentation: https://docs.oracle.com/javase/8/docs/
3. Java Swing Tutorial by Oracle: https://docs.oracle.com/javase/tutorial/uiswing/